Dynamic Programming Notes By Aman

Dynamic Programming (DP) is often considered one of the most intimidating topics in computer science, but at its core, it is simply **smart recursion**.

Here are your introductory notes to build a strong foundation before we dive into the specific patterns.

1. What is Dynamic Programming?

Dynamic Programming is an optimization technique. It is a method for solving complex problems by breaking them down into simpler subproblems, solving each of those subproblems just once, and storing their solutions.

Think of it as **"Recursion with a Notepad."**

- **Recursion:** You solve a big problem by breaking it into smaller pieces.
- **The Problem:** Standard recursion often solves the *same* small piece over and over again.
- **The DP Fix:** When you solve a small piece, you write the answer down (memoization). If you run into that same piece again, you look up the answer instead of re-solving it.

  Analogy: Imagine I ask you, "What is 1 + 1 + 1 + 1?" You count and say, "4." Now, I write "+ 1" on a piece of paper next to that equation. "What is it now?" You immediately say, "5." **Why?** You didn't re-count the first four 1s. You remembered (stored) that the previous sum was 4 and just added 1. **That is Dynamic Programming.**

2. Why is it used?

We use DP for one main reason: **Efficiency.**

Many problems, if solved using brute-force recursion, have an **Exponential Time Complexity** (e.g., $O(2^n)$). This is because they waste massive amounts of computing power solving the same sub-problems millions of times.

By applying DP, we reduce that complexity to **Polynomial Time** (e.g., $O(n)$ or $O(n^2)$). We trade a little bit of memory (to store the answers) for a massive gain in speed.

---

3. When to use Dynamic Programming? (The Checklist)

You should suspect a problem requires Dynamic Programming if it satisfies two main properties:

A. Overlapping Subproblems

The problem can be broken down into smaller subproblems which are reused several times.

- *Example:* In the Fibonacci sequence, to calculate `Fib(5)`, you need `Fib(4)` and `Fib(3)`. To calculate `Fib(4)`, you need `Fib(3)` and `Fib(2)`. Notice `Fib(3)` is needed twice. This is an overlap.
- *Counter-Example:* Merge Sort involves breaking a list into unique halves. The halves never overlap. That is Divide & Conquer, not DP.

B. Optimal Substructure

You can construct the optimal solution to the "big" problem using the optimal solutions of the "small" subproblems.

- *Example:* The shortest path from A to C might be the shortest path from A to B plus the path from B to C.

C. The "Keyword" Indicators

In coding interviews or competitive programming, look for these phrases in the problem statement. They are strong hints for DP:

- "Find the **maximum**..." (e.g., max profit, max subarray)
- "Find the **minimum**..." (e.g., min path cost, min coins)
- "Find the **longest**..." (e.g., longest common subsequence)
- "Count the **number of ways** to..." (e.g., ways to climb stairs)

The Roadmap: Pattern-Wise Learning

To master DP, you shouldn't just memorize problems; you must learn **patterns**. Here is the curriculum I have prepared for you, moving from Scratch to Advanced:

1. **Foundational Pattern:** Fibonacci Style (1D DP)
2. **Grid/Matrix Pattern:** Min/Max Path Sums (2D DP)
3. **Knapsack Patterns:** 0/1 Knapsack, Unbounded Knapsack
4. **String Patterns:** Longest Common Subsequence (LCS) & Palindromes
5. **Interval DP:** Matrix Chain Multiplication (MCM)
6. **Subset/Partition Patterns:** Subset Sum, Partition Equal Subset Sum
7. **DP on Trees:** Diameter of tree, Maximum path sum
8. **DP with Bitmasking:** Traveling Salesman Problem (Advanced)

This is the most critical skill in Dynamic Programming. If you can write the **Recursive Solution**, converting it to DP is just a matter of adding a few lines of code (memoization).

Let's break down exactly how to "think recursively" using our next pattern.

How to Think Recursively: The "Backward" Strategy

Don't think about how to start from index 0 and go forward. **Think backwards from the goal.**

The 3-Step Thought Process:

1. **Stand at the End:** Imagine you have already solved the problem and are standing at the very last step (the destination).
2. **Look Back One Step:** Ask yourself, "What was the *immediate* previous step I took to get here?"
3. **Make a Choice:** Usually, there are only a few possible previous steps. Your result depends on the best (min/max) or sum of those previous possibilities.

Pattern 1: Linear DP (Fibonacci Style)

This is the "Hello World" of Dynamic Programming. If you can master this, you understand the core mechanics of state transition.

1. How to Identify this Pattern

- **Dependency:** The value at the current step (i) depends only on a fixed number of previous steps (usually $i-1$ and $i-2$).
- **Question Type:**
  - "Count the number of ways to reach step N."
  - "Find the maximum money you can rob from a row of houses without robbing adjacent ones."
- **The "Choice":** At every step, you usually have a binary choice (e.g., take 1 step or 2 steps; rob this house or skip it).

2. Typical Questions

1. **Climbing Stairs:** You can take 1 or 2 steps at a time. How many ways to reach the top?
2. **House Robber:** You cannot rob adjacent houses. Maximize profit.
3. **Fibonacci Number:** Calculate the n-th Fibonacci number.
4. **Domino Tiling:** Ways to fill a $2 \times N$ board with 2×1 dominoes.

3. The Logic (Recurrence Relation)

The core formula usually looks like this:

$$dp[i] = dp[i-1] + dp[i-2]$$

(For counting ways)

Or like this:

$$dp[i] = \max(val[i] + dp[i-2], dp[i-1])$$

(For maximization problems like House Robber)

```java
class Solution {
    public int climbStairs(int n) {
        // Base cases
        if (n <= 1) return 1;

        // DP Array to store the solutions for each step
        // dp[i] represents the number of ways to reach step i
        int[] dp = new int[n + 1];

        // Initialization
        dp[0] = 1; // 1 way to be at ground (do nothing)
        dp[1] = 1; // 1 way to reach step 1 (1 step)

        // Iterate and fill the table
        for (int i = 2; i <= n; i++) {
            // The logic: To reach step i, you could have come from
            // step i-1 (by taking 1 step) OR step i-2 (by taking 2 steps)
            dp[i] = dp[i - 1] + dp[i - 2];
        }

        return dp[n];
    }
}
```

Pattern 2: Grid / Matrix DP (Min/Max Path)

Let's apply the "Recursive Thinking" strategy to this pattern.

The Problem: You are at the top-left cell `(0, 0)` of a grid. You want to reach the bottom-right cell `(m, n)`. You can only move **Down** or **Right**. Find the path with the **Minimum Cost**.

Step 1: Think Recursively (Backward)

- **Stand at the Goal:** I am at the bottom-right cell `(i, j)`.
- **Look Back:** How did I get here? Since I can only move Down or Right *to get here*, I must have arrived from:
  1. The cell **Above** me `(i-1, j)` (I moved Down).
  2. The cell to the **Left** of me `(i, j-1)` (I moved Right).
- **The Logic:** To minimize the cost to reach `(i, j)`, I should pick the minimum of the two paths leading to me, and add my own cost.

Step 2: The Recursive Line (Recurrence Relation)

This translates directly into math:

$$Cost(i, j) = \text{grid}[i][j] + \min(Cost(i-1, j),\ Cost(i, j-1))$$

- **Base Case:** If I am at `(0,0)`, the cost is just `grid[0][0]`.
- **Out of Bounds:** If `i < 0` or `j < 0`, return Infinity (so this path is never chosen).

Step 3: Pattern Identification

- **How to Identify:**
  - Input is a 2D Matrix or Grid.
  - Movement is restricted (e.g., "only right and down").
  - Objective involves "Minimum cost", "Maximum gold", or "Number of ways" to reach a corner.
- **Typical Questions:**
  - **Minimum Path Sum:** Find the path with the lowest sum.
  - **Unique Paths:** How many distinct paths exist from start to finish?
  - **Dungeon Game:** Find minimum health needed to survive a grid of threats.
  - **Maximal Square:** Find the largest square of 1s in a binary matrix.

```java
class Solution {
    // Memoization table to store results we've already calculated
    // dp[i][j] will store the min cost to reach cell (i, j)
    int[][] memo;

    public int minPathSum(int[][] grid) {
        int m = grid.length;
        int n = grid[0].length;
        memo = new int[m][n];

        // Initialize memo with -1 to indicate "not calculated yet"
        for(int[] row : memo) java.util.Arrays.fill(row, -1);

        // Start the recursive thought process from the END (bottom-right)
        return solve(grid, m - 1, n - 1);
    }
```

```java
    // The Recursive Function
    private int solve(int[][] grid, int i, int j) {
        // Base Case: We are at the start (top-left)
        if (i == 0 && j == 0) return grid[0][0];

        // Boundary Check: If we go off the grid (up or left), return Infinity
        // so this path is ignored by the Math.min function
        if (i < 0 || j < 0) return Integer.MAX_VALUE;

        // DP Check: If we already solved this subproblem, return the answer
        if (memo[i][j] != -1) return memo[i][j];

        // RECURSIVE LINE:
        // Calculate cost coming from Above (up)
        int up = solve(grid, i - 1, j);

        // Calculate cost coming from Left
        int left = solve(grid, i, j - 1);

        // The Logic: Current cost + min(path from up, path from left)
        memo[i][j] = grid[i][j] + Math.min(up, left);

        return memo[i][j];
    }
}
```

Why this works: The code looks exactly like our thought process. We ask for the answer at the end, and the computer recursively digs back to the start, solves the base case, and bubbles the answers back up, saving them in `memo` along the way.

Pattern 3: 0/1 Knapsack (The "Pick or Leave" Pattern)

The Problem: You are a thief with a knapsack that can hold a maximum weight `W`. There are `N` items in a room. Each item has a `weight` and a `value`. You want to maximize the total value in your bag.

The constraint is "0/1": You can either take the item whole (1) or leave it (0). You cannot break it.

Step 1: Think Recursively (The "Choice" Diagram)

Imagine you are standing at the last item (index `n-1`). You have a bag with some remaining capacity `W`. You have exactly two choices for this item:

1. **Don't Pick it (Exclude):** You decide this item is too heavy or not valuable enough. You skip it and move to the next item (index `n-2`) with your bag capacity unchanged.
2. **Pick it (Include):** You put it in your bag. You gain its `value`, but your bag capacity shrinks by its `weight`. You then move to the next item (index `n-2`) with less capacity.

Note: You can only pick the item if its weight is less than or equal to the current bag capacity.

Step 2: The Recursive Line (Recurrence Relation)

Our state needs two variables: the **index** of the item we are considering, and the **current capacity** of the bag.

$$f(i, w) = \max(\text{Don't Pick}, \text{Pick})$$

Where:

- $\text{Don't Pick} = f(i-1, w)$
- $\text{Pick} = \text{value}[i] + f(i-1, w - \text{weight}[i])$

Step 3: Pattern Identification

- **How to Identify:**
  - You are given a set of items with attributes (cost, value, weight).
  - You have a **constraint** or a target (max weight, target sum).
  - You need to **select a subset** of these items to maximize/minimize a value or count ways.
- **Typical Questions:**
  - **0/1 Knapsack:** Maximize value within weight limit.
  - **Subset Sum:** Is there a subset of numbers that sums up to `K`? (Here, "value" is ignored, we just care if capacity becomes 0).
  - **Partition Equal Subset Sum:** Can you split an array into two subsets with equal sums?

```java
class Solution {
    // dp[index][weight]
    int[][] memo;

    public int knapsack(int[] weights, int[] values, int capacity, int n) {
        // Initialize memoization table
        // Rows: items (0 to n), Cols: capacity (0 to capacity)
        memo = new int[n][capacity + 1];
        for (int[] row : memo) java.util.Arrays.fill(row, -1);

        // Start thinking from the last item
        return solve(weights, values, n - 1, capacity);
    }

    private int solve(int[] weights, int[] values, int index, int currentCap) {
        // Base Case: We ran out of items
        if (index < 0) {
            return 0;
        }

        // Base Case: If we are at the very first item (index 0)
        if (index == 0) {
            // If we have space, we steal it. Otherwise 0.
            if (weights[0] <= currentCap) return values[0];
            return 0;
        }

        // Return if already calculated
        if (memo[index][currentCap] != -1) return memo[index][currentCap];

        // CHOICE 1: DON'T PICK
        // Move to index-1, capacity stays same
        int notPick = 0 + solve(weights, values, index - 1, currentCap);

        // CHOICE 2: PICK
        // Move to index-1, capacity reduces by weights[index]
        // Add values[index] to our profit
        int pick = Integer.MIN_VALUE;
        if (weights[index] <= currentCap) {
            pick = values[index] + solve(weights, values, index - 1, currentCap - weights[index]);
        }

        // Store and return the max of the two choices
        return memo[index][currentCap] = Math.max(pick, notPick);
    }
}
```

Crucial Variation: Unbounded Knapsack

Before we move to strings, you must know a slight twist on this pattern.

- **0/1 Knapsack:** Item can be chosen **once** or not at all.
- **Unbounded Knapsack:** Item can be chosen **unlimited** times.

Recursive Change: If you pick an item, you stay at the **same index** (so you can pick it again), instead of moving to `index - 1`.

- *Code change:* `solve(index - 1, ...)` becomes `solve(index, ...)` in the "Pick" step.
- *Example:* **Coin Change Problem** (Find min coins to make amount X).

Pattern 4: String DP (Longest Common Subsequence - LCS)

The Problem: Given two strings, `text1` and `text2`, find the length of their longest common subsequence.

- *Example:* `text1 = "abcde"`, `text2 = "ace"`
- *Result:* 3 ("ace" is the subsequence).

Unlike a substring, a **subsequence** doesn't need to be consecutive, but the order must remain the same.

Step 1: Think Recursively (The "Matching" Logic)

Imagine you have two pointers, `i` pointing to the end of `text1` and `j` pointing to the end of `text2`.

You compare the characters `text1[i]` and `text2[j]`. There are only two possibilities:

1. **They Match:** Great! You found a common character. Add 1 to your result, and move **both** pointers back (`i-1` , `j-1`) to look for more matches.
2. **They Don't Match:** You have a choice. You can't match these two current characters, so you must either:
   - Skip the character in `text1` (move `i` back) and keep `j`.
   - Skip the character in `text2` (move `j` back) and keep `i`.
   - You take the **maximum** result of these two options.

Step 2: The Recursive Line

$$dp(i,j) = \begin{cases} 1 + dp(i-1, j-1) & \text{if } text1[i] == text2[j] \\ \max(dp(i-1,j), dp(i,j-1)) & \text{if } text1[i] \neq text2[j] \end{cases}$$

Step 3: Pattern Identification

- **How to Identify:**
  - Input involves **two strings** (or arrays).
  - Problem asks for "longest common...", "minimum edits", "shortest supersequence".
- **Typical Questions:**
  - **Longest Common Subsequence (LCS):** The base problem.
  - **Longest Common Substring:** Characters must be consecutive (reset count to 0 if no match).
  - **Edit Distance:** Min operations (insert, delete, replace) to convert String A to B.
  - **Longest Palindromic Subsequence:** Find LCS of string `S` and `Reverse(S)`.

```java
class Solution {
    int[][] memo;

    public int longestCommonSubsequence(String text1, String text2) {
        int m = text1.length();
        int n = text2.length();
        memo = new int[m][n];

        for(int[] row : memo) java.util.Arrays.fill(row, -1);

        // Start from the last characters of both strings
        return solve(text1, text2, m - 1, n - 1);
    }

    private int solve(String s1, String s2, int i, int j) {
        // Base Case: If either string runs out, no more common chars possible
        if (i < 0 || j < 0) return 0;

        if (memo[i][j] != -1) return memo[i][j];

        // OPTION 1: Characters Match
        if (s1.charAt(i) == s2.charAt(j)) {
            // Add 1 and move both indices back
            return memo[i][j] = 1 + solve(s1, s2, i - 1, j - 1);
        }

        // OPTION 2: Characters Don't Match
        else {
            // Try skipping char from s1, or skipping char from s2
            // Take the max of the two outcomes
            return memo[i][j] = Math.max(
                solve(s1, s2, i - 1, j),
                solve(s1, s2, i, j - 1)
            );
        }
    }
}
```

Pattern 5: Interval DP (Merging / MCM)

This pattern is slightly more advanced. Instead of moving from left to right (0 to N), you solve for every possible **interval** (i to j).

The Problem: Matrix Chain Multiplication (MCM). Given a sequence of matrices, find the most efficient way to multiply them (minimize operations).

Step 1: Think Recursively (The "Partition" Logic)

Imagine you have a range `[i, j]`. You want to solve the problem for this whole range. To do that, you must try every possible place `k` to **split** this range into two halves:

- Left half: `[i, k]`
- Right half: `[k+1, j]`

You solve the left half, solve the right half, and then combine them. You try *all* possible `k` positions and pick the best one.

Step 2: The Recursive Line

$$dp(i,j) = \min_{k=i}^{j-1}\{dp(i,k) + dp(k+1,j) + \text{cost to combine}\}$$

Step 3: Pattern Identification

- **How to Identify:**
  - Input is an array.
  - You are asked to minimize/maximize a value by combining elements.
  - The order of elements cannot be changed (unlike sorting), but the order of *operations* matters.
- **Typical Questions:**
  - **Matrix Chain Multiplication:** Minimize scalar multiplications.
  - **Burst Balloons:** Maximize coins by bursting balloons (tricky interval logic).
  - **Minimum Cost to Cut a Stick:** Choose cut order to minimize cost.
  - **Palindrome Partitioning:** Min cuts to make every substring a palindrome.

```java
class Solution {
    int[][] memo;

    public int solveInterval(int[] arr, int i, int j) {
        // Base Case: An interval of size 1 cannot be multiplied/combined
        if (i >= j) return 0;

        if (memo[i][j] != -1) return memo[i][j];

        int minCost = Integer.MAX_VALUE;

        // Try every possible split point 'k' between i and j
        for (int k = i; k < j; k++) {
            // Recurse Left + Recurse Right + Cost to Multiply the two resulting matrices
            int cost = solveInterval(arr, i, k)
                     + solveInterval(arr, k + 1, j)
                     + (arr[i-1] * arr[k] * arr[j]); // This 'cost' formula changes per problem

            minCost = Math.min(minCost, cost);
        }

        return memo[i][j] = minCost;
    }
}
```

We have covered the major pillars.

6. **Subset / Partition Patterns:** (Similar to Knapsack but focused on sums).
7. **DP on Trees:** (Using tree traversal recursion).
8. **DP with Bitmasking:** (Advanced state representation for small N).

Pattern 6: DP on Trees (The "Post-Order" Pattern)

This pattern is very common in interviews because trees are naturally recursive. If you know how to write a DFS (Depth First Search), you are 90% there.

The Problem: You are given a binary tree. You need to find the **Maximum Path Sum** (path can start and end anywhere) or the **Diameter** (longest path between any two nodes).

Step 1: Think Recursively (The "Bubbling Up" Logic)

Think about a single node `root`. It has a left child and a right child. To solve the problem for the `root`, you first need the answers from `left` and `right`. This implies a **Post-Order Traversal** (Left -> Right -> Root).

Imagine you are the `root` node. You ask your children:

1. "Hey Left Child, what is the best path ending at you?"
2. "Hey Right Child, what is the best path ending at you?"
3. **The Calculation:** Once they answer, you calculate the answer passing through *yourself*.

Step 2: The Recursive Line

For "Binary Tree Maximum Path Sum": At any node `u`, there are two things to track:

1. **The Max Passing Through Me:** `LeftMax + RightMax + NodeValue`. (This could be the global maximum answer).
2. **The Max I Return to My Parent:** `NodeValue + max(LeftMax, RightMax)`. (I can only extend one branch up to my parent).

Step 3: Pattern Identification

- **How to Identify:**
  - Input is a **Tree** (Binary Tree or N-ary Tree).
  - Questions like "Longest path", "Max sum path", "Vertex Cover", "House Robber on Tree".
  - You cannot solve it just by looking at the top; you need info from the bottom leaves up.

```java
class Solution {
    int globalMax = Integer.MIN_VALUE;

    public int maxPathSum(TreeNode root) {
        solve(root);
        return globalMax;
    }

    // This function returns the max path sum "starting" from the current node
    // and going downwards into one of its subtrees.
    private int solve(TreeNode node) {
        if (node == null) return 0;

        // Recursively ask left and right children
        // Math.max(0, ...) is important! If a child returns a negative path,
        // we ignore it (treat it as 0) because it would only decrease our sum.
        int leftGain = Math.max(0, solve(node.left));
        int rightGain = Math.max(0, solve(node.right));

        // INTERNAL CALCULATION:
        // Check if the path curving through THIS node is the best path ever found.
        // (Left -> Node -> Right)
        int currentPathSum = node.val + leftGain + rightGain;
        globalMax = Math.max(globalMax, currentPathSum);

        // RETURN TO PARENT:
        // We can only carry ONE branch up to the parent. Pick the heavier one.
        return node.val + Math.max(leftGain, rightGain);
    }
}
```

Pattern 7: DP with Bitmasking (The "Small N" Pattern)

This is considered an "Advanced" pattern, often seen in hard interview questions or competitive programming.

The Problem: You have `N` cities. You must visit every city exactly once and return to the start. Find the minimum cost (Traveling Salesman Problem). Or, you have `N` tasks and `N` people; assign tasks to minimize cost.

The key constraint: **N is very small (usually N <= 20).**

Step 1: Think Recursively (The "Set" Logic)

In standard DP, our state is usually an index `i`. Here, "index" isn't enough. If I am at City 3, I need to know **"Which cities have I already visited?"**

We cannot use a `HashSet` or `List` inside our DP state (it's too slow/complex). Instead, we use an **Integer** as a binary string to represent the set.

- If `N=4`, and we have visited city 0 and city 2:
- Mask = `0101` (binary) = `5` (decimal).
- The k-th bit is 1 if City k is visited.

Step 2: The Recursive Line

State: `dp(mask, currentCity)`

- `mask` : Represents the set of visited cities.
- `currentCity` : Where we are right now.

Transition: Try to go to every neighbor `nextCity` that is **not** in the `mask`.

$$dp(\text{mask}, u) = \min(\text{cost}(u, v) + dp(\text{mask} \mid (1 \ll v),\ v))$$

(Note: `|` is the bitwise OR operator to mark `v` as visited)

Step 3: Pattern Identification

- **How to Identify:**
  - **N is small (<= 20).** This is the biggest hint. $2^{20} \approx 10^6$, which fits in time limits.
  - Problem involves "visiting all nodes", "matching pairs", or "ordering items".
  - Keywords: "Hamiltonian Path", "Assignments".

```java
class Solution {
    int[][] dist;
    int n;
    int[][] memo;

    public int shortestPath(int[][] graph) {
        this.dist = graph;
        this.n = graph.length;
        // 2^n states for mask, n states for current city
        this.memo = new int[1 << n][n];

        for(int[] row : memo) java.util.Arrays.fill(row, -1);

        // Start at city 0, with mask '1' (meaning city 0 is visited)
        return solve(1, 0);
    }

    private int solve(int mask, int currentCity) {
        // Base Case: If all cities are visited (mask is all 1s)
        // Return cost to go back to start (City 0)
        if (mask == (1 << n) - 1) {
            return dist[currentCity][0];
        }

        if (memo[mask][currentCity] != -1) return memo[mask][currentCity];

        int minCost = 10000000; // A large number (Infinity)

        // Try visiting every other city
        for (int nextCity = 0; nextCity < n; nextCity++) {

            // Check if nextCity is NOT visited yet in the mask
            if ((mask & (1 << nextCity)) == 0) {

                // New Mask: Turn on the bit for nextCity
                int newMask = mask | (1 << nextCity);

                int cost = dist[currentCity][nextCity] + solve(newMask, nextCity);
                minCost = Math.min(minCost, cost);
            }
        }

        return memo[mask][currentCity] = minCost;
    }
}
```

Summary of the Master Plan

You now have the complete toolkit. Here is how to practice them in order:

1. **Linear DP:** Climbing Stairs, House Robber.
2. **Grid DP:** Min Path Sum, Unique Paths.
3. **Knapsack:** 0/1 Knapsack, Coin Change (Unbounded).
4. **Strings:** LCS, Edit Distance.
5. **Intervals:** MCM, Burst Balloons.
6. **Trees:** Max Path Sum, Diameter.
7. **Bitmask:** Traveling Salesman (TSP).

Pattern 8: Digit DP (Counting Numbers)

This is a very specific, template-based pattern. It is used when a problem asks you to count integers within a range `[L, R]` that satisfy a certain property.

The Problem: Find the count of numbers between `0` and `N` that do not contain the digit `4`.

- *Brute Force:* Iterate from 0 to N, check every number. (Too slow if $N = 10^{18}$).
- *Digit DP:* Construct the number digit by digit from left to right.

Step 1: Think Recursively (The "Tight" Constraint)

Imagine you are building a number digit by digit. If $N = 256$, and you are deciding the first digit (hundreds place):

- You can place `0`, `1`, or `2`.
- **If you place `0` or `1`:** You are strictly below the prefix of N. For the next digits, you are free to place anything (`0-9`). The constraint is "loose".
- **If you place `2`:** You are matching the prefix of N. For the next digit, you are restricted. You can only place `0` to `5`. This is the **Tight Constraint**.

Step 2: The Recursive Line

The state usually requires:

1. `index` : Which digit position (left to right) we are filling.
2. `tight` : Boolean. `true` if we are restricted by the digits of N, `false` if we are free.
3. `state` : Any other condition (e.g., previous digit, sum of digits, count of non-zero digits).

$$dp(index, tight, state) = \sum_{digit=0}^{limit} dp(index + 1,\ tight\&\&(digit == limit),\ \text{newState})$$

Step 3: Pattern Identification

- **How to Identify:**
  - Input is a large number range (e.g., 1 to 10^{18}).
  - Question asks to count integers with properties like "sum of digits equals X", "digits are non-decreasing", "does not contain 4".
- **Typical Questions:**
  - **Non-negative Integers without Consecutive Ones:** Count numbers $\leq N$ binary representation.

- **Digit 1:** Count how many times digit 1 appears in numbers $\leq N$.

```java
class Solution {
    String s; // String representation of N
    Long[][][] memo;

    public int countNumbersWithout4(int n) {
        s = String.valueOf(n);
        // index (length of number), tight (0 or 1), has4 (0 or 1 - optional based on problem)
        memo = new Long[s.length()][2][2];
        return (int) solve(0, true, false);
    }

    // index: current digit position (0 to length-1)
    // tight: true if we are restricted by N's digits
    // has4: true if we have already placed a '4' (problem specific logic)
    private long solve(int index, boolean tight, boolean has4) {
        if (index == s.length()) {
            return 1; // We successfully built one valid number
        }

        // Convert boolean to int for memoization index
        int tightIndex = tight ? 1 : 0;
        int has4Index = has4 ? 1 : 0;
        if (memo[index][tightIndex][has4Index] != null) return memo[index][tightIndex][has4Index];

        long ans = 0;

        // The upper bound for this digit.
        // If tight, we can only go up to s.charAt(index). Otherwise 9.
        int limit = tight ? (s.charAt(index) - '0') : 9;

        for (int digit = 0; digit <= limit; digit++) {
            // Check problem specific constraint (e.g., skip 4)
            if (digit == 4) continue;

            // Update tight constraint for the next position
            // Next is tight ONLY IF current is tight AND we picked the limit digit
            boolean nextTight = tight && (digit == limit);

            ans += solve(index + 1, nextTight, has4); // has4 logic would go here if needed
        }

        return memo[index][tightIndex][has4Index] = ans;
    }
}
```

Pattern 9: State Machine DP (Stock Trading)

This pattern is a subset of linear DP, but it requires drawing a **Finite State Machine (FSM)** diagram to solve. It is heavily used in "Stock Buy and Sell" problems.

The Problem: Best Time to Buy and Sell Stock with Cooldown. You have an array of prices. You can buy/sell multiple times, but after selling, you must rest (cooldown) for 1 day.

Step 1: Think Recursively (The State Diagram)

Instead of just `index`, think about "What state of ownership am I in?"

1. **Held:** I have a stock in my hand.
2. **Sold:** I just sold a stock today.
3. **Reset (No Stock):** I have no stock and I am ready to buy.

You map out the transitions:

- From **Reset** $\rightarrow$ Buy $\rightarrow$ **Held**.
- From **Held** $\rightarrow$ Sell $\rightarrow$ **Sold**.
- From **Sold** $\rightarrow$ Wait $\rightarrow$ **Reset**.

Step 2: The Recursive Line

We usually maintain 3 arrays (or variables) representing the max profit in each state at day `i`.

- `Held[i]` = Max of (Keep holding from yesterday, OR Buy today from Reset state)

$$Held[i] = \max(Held[i-1],\ Reset[i-1] - prices[i])$$

- `Sold[i]` = Sell what we held yesterday

$$Sold[i] = Held[i-1] + prices[i]$$

- `Reset[i]` = Max of (Keep resting, OR Just finished cooldown from Sold)

$$Reset[i] = \max(Reset[i-1],\ Sold[i-1])$$

Step 3: Pattern Identification

- **How to Identify:**
  - Problem involves complex rules about changing states over time.
  - "Cooldown", "Transaction fees", "Can only complete k transactions".
  - Input is a linear array (usually time-series data).

```java
class Solution {
    public int maxProfit(int[] prices) {
        if (prices.length <= 1) return 0;

        // Initialize states for Day 0
        int held = -prices[0]; // We bought stock, so profit is negative
        int sold = 0;          // Can't sell on day 0
        int reset = 0;         // Starting state

        for (int i = 1; i < prices.length; i++) {
            // Store previous values to calculate new ones simultaneously
            int prevHeld = held;
            int prevSold = sold;
            int prevReset = reset;

            // 1. Held State: Either kept holding or bought new stock today
            held = Math.max(prevHeld, prevReset - prices[i]);

            // 2. Sold State: Sold the stock we were holding
            sold = prevHeld + prices[i];

            // 3. Reset State: Either kept resting or cooled down from a sale
            reset = Math.max(prevReset, prevSold);
        }

        // The result is max of Sold or Reset (we can't end while Holding stock)
        return Math.max(sold, reset);
    }
}
```

This is the curated list of the most famous and high-yield practice questions for each pattern. If you master these, you will be able to solve 90% of the DP problems asked in interviews.

I have arranged them broadly in order of difficulty within each category.

1. Linear DP (Fibonacci & 1D)

Focus: State depends on i-1, i-2.

1. **Climbing Stairs** (LeetCode 70) - *The absolute basic.*
2. **House Robber** (LeetCode 198) - *Classic "skip or take" logic.*
3. **House Robber II** (LeetCode 213) - *Same as above, but houses are in a circle.*
4. **Decode Ways** (LeetCode 91) - *Handling edge cases with '0'.*
5. **Longest Increasing Subsequence** (LeetCode 300) - *Can be solved in $O(N^2)$ with DP or $O(N \log N)$ with Binary Search.*

2. Grid / Matrix DP

Focus: Minimum cost or ways to reach a cell from Top/Left.

1. **Unique Paths** (LeetCode 62) - *Counting ways to reach corner.*
2. **Unique Paths II** (LeetCode 63) - *Grid with obstacles.*
3. **Minimum Path Sum** (LeetCode 64) - *Min cost path.*
4. **Maximal Square** (LeetCode 221) - *Find largest square of 1s (requires smart recurrence).*
5. **Dungeon Game** (LeetCode 174) - *Hard. Requires thinking backwards from destination to start.*

3. Knapsack Patterns (0/1 & Unbounded)

Focus: Subset selection under constraints.

1. **Partition Equal Subset Sum** (LeetCode 416) - *Can you split array into two equal sums?*
2. **Coin Change** (LeetCode 322) - *Min coins to make amount (Unbounded).*
3. **Coin Change II** (LeetCode 518) - *Number of ways to make amount (Unbounded).*
4. **Target Sum** (LeetCode 494) - *Assign + or - to numbers to get target.*
5. **Ones and Zeroes** (LeetCode 474) - *2D Knapsack (two constraints: max 0s and max 1s).*

4. String DP (LCS & Palindromes)

Focus: Two pointers comparing characters.

1. **Longest Common Subsequence** (LeetCode 1143) - *The mother of all string DP.*
2. **Edit Distance** (LeetCode 72) - *Min operations to convert Word A to Word B.*
3. **Longest Palindromic Subsequence** (LeetCode 516) - *LCS on string S and reverse(S).*
4. **Distinct Subsequences** (LeetCode 115) - *Count ways string B occurs in string A.*
5. **Wildcard Matching** (LeetCode 44) - *Complex pattern matching with* `*` *and* `?`.

5. Interval DP (Merge / Split)

Focus: Optimal way to split range [i...j].

1. **Matrix Chain Multiplication** (Classic / GFG) - *Standard textbook problem.*
2. **Longest Palindromic Substring** (LeetCode 5) - *Can be done with expansion, but DP is instructive.*
3. **Burst Balloons** (LeetCode 312) - *Tricky. Requires thinking in reverse order of bursting.*
4. **Minimum Cost to Cut a Stick** (LeetCode 1547) - *Similar to MCM.*
5. **Palindrome Partitioning II** (LeetCode 132) - *Min cuts to make all parts palindromes.*

6. DP on Trees

Focus: Post-order traversal (Bottom-Up).

1. **Diameter of Binary Tree** (LeetCode 543) - *Longest path between any two nodes.*
2. **Binary Tree Maximum Path Sum** (LeetCode 124) - *Hard. Path can start/end anywhere.*
3. **House Robber III** (LeetCode 337) - *Thief on a tree structure.*
4. **Unique Binary Search Trees** (LeetCode 96) - *Catalan Numbers.*
5. **Binary Tree Cameras** (LeetCode 968) - *Min cameras to cover all nodes (Greedy/DP).*

7. Bitmask DP (Small N)

Focus: Representing visited sets as integers.

1. **Partition to K Equal Sum Subsets** (LeetCode 698) - *Can be solved with Backtracking or Bitmask.*
2. **Traveling Salesman Problem** (Classic) - *Often appears as "Find Shortest Superstring" or similar variations.*
3. **Beautiful Arrangement** (LeetCode 526) - *Permutations satisfying conditions.*
4. **Smallest Sufficient Team** (LeetCode 1125) - *Advanced. Selecting team to cover all skills.*
5. **Maximum Students Taking Exam** (LeetCode 1349) - *Grid + Bitmask (broken profile DP).*

8. Digit DP

Focus: Counting numbers with specific properties.

1. **Number of Digit One** (LeetCode 233) - *Count 1s in 0..N.*
2. **Non-negative Integers without Consecutive Ones** (LeetCode 600) - *Standard constraint problem.*
3. **Numbers At Most N Given Digit Set** (LeetCode 902).
4. **Rotated Digits** (LeetCode 788) - *Valid numbers after 180-degree rotation.*
5. **Digit Count in Range** (General Competitive Programming) - *Count occurrences of digit d in range [L, R].*

9. State Machine DP

Focus: Managing specific states (Hold, Sold, Wait).

1. **Best Time to Buy and Sell Stock with Cooldown** (LeetCode 309).
2. **Best Time to Buy and Sell Stock III** (LeetCode 123) - *At most 2 transactions.*
3. **Best Time to Buy and Sell Stock IV** (LeetCode 188) - *At most K transactions.*
4. **Maximum Subarray Sum with One Deletion** (LeetCode 1186).
5. **Wiggle Subsequence** (LeetCode 376) - *Longest alternating subsequence.*